

Задача №10: Решение нелинейных уравнений методом Чебышева

1 Введение

В данной задаче рассматривается численное решение нелинейных уравнений вида $f(x) = 0$ с использованием итерационного метода Чебышева высокого порядка. Этот метод обеспечивает более высокую скорость сходимости по сравнению со стандартным методом Ньютона за счет вычисления старших производных функции.

В работе реализован алгоритм для решения двух модельных функций:

1. $f(x) = x^m - a = 0$ (вычисление корня m -й степени: $x^* = \sqrt[m]{a}$).
2. $f(x) = e^x - a = 0$ (вычисление логарифма: $x^* = \ln(a)$).

2 Теоретическая база: Разложение обратной функции

Теория. Пусть задано нелинейное уравнение $f(x) = 0$, и мы ищем его точный корень z , то есть $f(z) = 0$.

Рассмотрим обратную функцию $x = F(y)$, по определению удовлетворяющую условию $F(f(x)) = x$. Если мы сможем найти значение обратной функции в нуле, мы точно найдем искомым корень:

$$z = F(0)$$

Пусть x_n — текущее приближение к корню, а значение функции в этой точке равно $y_n = f(x_n)$. Разложим функцию $F(y)$ в ряд Тейлора в окрестности известной нам точки y_n , сохраняя члены вплоть до пятой производной (для обеспечения пятого порядка сходимости, используемого в программе):

$$F(y) = F(y_n) + F'(y_n)(y - y_n) + \frac{F''(y_n)}{2!}(y - y_n)^2 + \dots + \frac{F^{(5)}(y_n)}{5!}(y - y_n)^5 + O((y - y_n)^6)$$

Так как мы ищем $z = F(0)$, подставим $y = 0$ в это разложение. Учитывая, что $F(y_n) = x_n$, получаем итерационную формулу:

$$x_{n+1} = x_n - F'(y_n)y_n + \frac{F''(y_n)}{2}y_n^2 - \frac{F'''(y_n)}{6}y_n^3 + \frac{F^{(4)}(y_n)}{24}y_n^4 - \frac{F^{(5)}(y_n)}{120}y_n^5$$

Чтобы использовать эту формулу на практике, необходимо выразить производные обратной функции $F(y)$ через производные исходной функции $f(x)$, последовательно применяя правило дифференцирования сложной функции $\left(\frac{dx}{dy} = \frac{1}{f'(x)}\right)$:

$$\begin{aligned} F'(y) &= \frac{1}{f'(x)} \\ F''(y) &= \frac{d}{dy} \left(\frac{1}{f'(x)} \right) = \frac{dx}{dy} \cdot \frac{d}{dx} \left(\frac{1}{f'(x)} \right) = -\frac{f''(x)}{(f'(x))^3} \\ F'''(y) &= \frac{d}{dy} \left(-\frac{f''(x)}{(f'(x))^3} \right) = \frac{3(f''(x))^2 - f'(x)f'''(x)}{(f'(x))^5} \end{aligned}$$

Уже на третьей производной формула становится громоздкой. Вычисление $F^{(4)}(y)$ и $F^{(5)}(y)$ в общем виде порождает длинные алгебраические дроби с комбинациями производных $f(x)$ вплоть до пятого порядка.

Именно поэтому вычисление таких поправок «в лоб» на каждой итерации вычислительно крайне неэффективно. Однако для конкретных функций (таких как $x^m - a$ и $e^x - a$) можно обойти прямое вычисление этих производных, построив эквивалентное разложение аналитически.

3 Вывод коэффициентов для модельных функций

Для применения разложения на практике введем безразмерный параметр малости (нормированную невязку) t , зависящий от текущего шага x_n .

3.1 Функция $f(x) = x^m - a = 0$

Нам необходимо найти корень $z = \sqrt[m]{a} = a^{1/m}$. Введем параметр t , характеризующий относительное отклонение текущего приближения:

$$t = 1 - \frac{a}{x_n^m} \implies \frac{a}{x_n^m} = 1 - t \implies a = x_n^m(1 - t)$$

Подставим это выражение для a в формулу точного корня:

$$z = a^{1/m} = (x_n^m(1 - t))^{1/m} = x_n(1 - t)^{1/m}$$

Так как вблизи корня $x_n^m \approx a$, параметр $t \rightarrow 0$. Разложим выражение $(1 - t)^{1/m}$ в ряд Маклорена (обобщенный биномиальный ряд Ньютона) по степеням t :

$$(1 - t)^\alpha = 1 - \alpha t + \frac{\alpha(\alpha - 1)}{2!}t^2 - \frac{\alpha(\alpha - 1)(\alpha - 2)}{3!}t^3 + \dots$$

Полагая $\alpha = \frac{1}{m}$, последовательно вычислим коэффициенты c_k при степенях t^k вплоть до пятого порядка:

$$\begin{aligned} c_1 &= -\alpha = -\frac{1}{m} \\ c_2 &= \frac{\alpha(\alpha - 1)}{2} = \frac{\frac{1}{m}(\frac{1}{m} - 1)}{2} = \frac{1 - m}{2m^2} \\ c_3 &= -\frac{\alpha(\alpha - 1)(\alpha - 2)}{6} = -\frac{\frac{1}{m}(\frac{1-m}{m})(\frac{1-2m}{m})}{6} = -\frac{(1 - m)(1 - 2m)}{6m^3} \\ c_4 &= \frac{\alpha(\alpha - 1)(\alpha - 2)(\alpha - 3)}{24} = \frac{(1 - m)(1 - 2m)(1 - 3m)}{24m^4} \\ c_5 &= -\frac{\alpha(\alpha - 1)(\alpha - 2)(\alpha - 3)(\alpha - 4)}{120} = -\frac{(1 - m)(1 - 2m)(1 - 3m)(1 - 4m)}{120m^5} \end{aligned}$$

Таким образом, итерационная формула принимает мультипликативный вид:

$$x_{n+1} = x_n (1 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5)$$

3.2 Функция $f(x) = e^x - a = 0$

Нам необходимо найти корень $z = \ln(a)$. Аналогично введем параметр невязки t :

$$t = 1 - \frac{a}{e^{x_n}} \implies \frac{a}{e^{x_n}} = 1 - t \implies a = e^{x_n}(1 - t)$$

Подставим это выражение для a в формулу точного корня, используя свойства логарифма:

$$z = \ln(a) = \ln(e^{x_n}(1 - t)) = \ln(e^{x_n}) + \ln(1 - t) = x_n + \ln(1 - t)$$

Разложим функцию $\ln(1 - t)$ в ряд Тейлора (ряд Меркатора), справедливый при $|t| < 1$:

$$\ln(1 - t) = -t - \frac{t^2}{2} - \frac{t^3}{3} - \frac{t^4}{4} - \frac{t^5}{5} - \dots$$

Коэффициенты разложения c_k при степенях t^k здесь являются строгими константами:

$$c_1 = -1, \quad c_2 = -\frac{1}{2}, \quad c_3 = -\frac{1}{3}, \quad c_4 = -\frac{1}{4}, \quad c_5 = -\frac{1}{5}$$

Итерационная формула принимает аддитивный вид:

$$x_{n+1} = x_n + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5$$

4 Реализация алгоритма

Алгоритм реализован на языке C++ с разбиением на интерфейс (`chebyshev.hpp`) и реализацию (`chebyshev.cpp`). Сборка осуществляется с помощью утилиты `make` со строгими флагами компиляции (`-O3 -Wall -Wextra -Werror`). Для оптимизации вычисления полинома 5-й степени в коде применена вычислительная **схема Горнера**, сокращающая количество ресурсоемких операций умножения.

4.1 Критерии останова

Итерационный процесс продолжается до тех пор, пока не выполнится одно из следующих условий:

1. Достигнуто максимальное число итераций (`MAX_ITER = 1000`).
2. Малость приращения (сходимость по аргументу): $|x_{n+1} - x_n| < \varepsilon$.
3. Малость невязки (сходимость по значению): $|f(x_{n+1})| < \varepsilon$.

В программе задана точность $\varepsilon = 10^{-10}$, что обеспечивает нахождение корня с высокой достоверностью.

4.2 Численный эксперимент

Для проверки работоспособности алгоритма были проведены тесты на обеих модельных функциях. Программа продемонстрировала феноменальную скорость сходимости:

- Для функции $f(x) = e^x - a$: алгоритм сошелся к корню $x^* \approx 1.3862943611$ всего за **7 итераций**. Итоговая невязка составила $\approx 8.88 \times 10^{-16}$.
- Для функции $f(x) = x^m - a$: алгоритм сошелся к корню $x^* \approx 1.2190136542$ за **8 итераций**. Итоговая невязка составила $\approx 1.33 \times 10^{-15}$.

5 Выводы

За счет предварительного аналитического вывода коэффициентов разложения (разложения обратной функции $F(0) = z$) мы можем блестяще адаптировать метод Чебышева для конкретных нелинейных функций, избегая дорогостоящего численного вычисления производных на каждом шаге итерации.

Сохранение членов ряда вплоть до 5-го порядка (t^5) в программной реализации обеспечивает сверхбыструю сходимость. Как показал численный эксперимент, метод позволяет достигать **машинного нуля** (машинной точности $\varepsilon_{mach} \approx 10^{-16}$ для типа `double`) за единицы итераций. Использование метода Чебышева высокого порядка делает алгоритм сверхустойчивым и предпочтительным по сравнению со стандартным методом Ньютона при работе с экспоненциальными и степенными функциями.